

How Content Delivery Network and Reverse Proxy Mechanism from Cloudflare Accelerate the Web

Name: HAN Zhuo

Student ID: 60130387

Date: November 21, 2025

1 Introduction

Modern web applications and video services increasingly rely on **content delivery networks** (CDNs, [8]) and edge nodes to cope with high latency, cross-continent traffic, and rapidly growing workloads. Instead of sending every HTTP request directly to a single origin server, many popular sites now use large, globally distributed platforms that cache content [12], terminate TLS connections, and route traffic on behalf of the origin. Cloudflare [6, 9, 10] is one of the most visible examples of such a platform. It operates a worldwide CDN, an anycast-based edge network, and an authoritative DNS [11] service that together sit in front of millions of origin websites and their end users.

I first became aware of Cloudflare’s role during a noticeable outage on 18 November 2025 [3, 1]. When I tried to open `tetris.com`, the browser showed an error page with a 500 status code and three machine icons: my local device, a Cloudflare server, and the Tetris origin server. The Cloudflare icon was highlighted in red, and several other unrelated sites showed similar Cloudflare-branded error pages. This made it clear that many websites were relying on the same intermediate service and that a failure at this layer could affect them all at once. Before this event, I had implicitly thought of web access as a simple client–server connection. The outage showed that there was an additional third-party layer in between.

This report uses Cloudflare as a concrete example to see how modern CDNs are placed inside today’s Internet architecture. The discussion focuses on its behaviour as a reverse proxy and CDN across the application and network layers and relates these behaviours to concepts from Chapters 1–7 of the textbook, including HTTP, DNS, routing, and web caching [12].

2 Background

2.1 Basic Client-Server and HTTP

In the basic web model, a user opens a browser and types a URL. The browser works as an HTTP client and sends a request to the web server that hosts the site [12]. The server processes the request, generates a response (for example an HTML page), and sends it back over TCP. This simple client–server pattern is the main example in the application-layer chapter of the textbook, and it is still the starting point for most web applications today.

However, this direct model has some clear problems. If the server is far away, the round-trip time (RTT) is high and every request feels slow. When many users access the same popular site at the same time, the single origin server can become overloaded, which leads to high delay or even failure. If the only server goes down, the whole service becomes unavailable. These limitations are the main reasons why extra layers, such as CDNs and proxies, were added on top of the basic client–server design.

2.2 Content Delivery Networks (CDN)

A **content delivery network** (CDN) is a group of cache servers that are placed at many locations around the world. Instead of always fetching data from the single origin server, users can download copies of images, scripts, or even full pages from a nearby cache node. From the user’s point of view, they still send an HTTP request as usual, but in many cases the request is handled by a CDN node that is geographically closer and less loaded than the origin.

This idea directly targets the delay and scalability issues of the basic model. By storing popular content at the edge, CDNs reduce RTT and save bandwidth on long-distance links [2]. They also protect the origin server from having to answer every single request. This idea matches the textbook discussion of web caching at the application layer, where content is moved closer to users in order to reduce response time.

2.3 Forward and Reverse Proxy

Proxies are another important building block. A forward proxy sits on the client side. It receives requests from users and then sends requests to different websites on their behalf. Enterprise networks often deploy forward proxies (such as Squid) for access control, logging, or shared caching for many employees.

A reverse proxy sits on the server side. It receives requests from the public Internet and then forwards them to one or more origin servers inside a private network. To the client, the reverse proxy looks like the actual website. It can hide the internal structure of the service, distribute requests across multiple back-end machines, and cache responses. Cloudflare mainly acts as this type of reverse proxy: it accepts connections from users, applies its own logic (caching, TLS termination, optimisation), and only then contacts the origin server.

3 Cloudflare Architecture

3.1 DNS and Anycast

To understand where Cloudflare fits in, it is helpful to follow what happens when a user visits a site that is behind Cloudflare, as illustrated in Figure 1. Suppose the user types `https://example.com` in the browser [6]. First, the browser asks a local DNS resolver (usually provided by the ISP



Figure 1: DNS resolution for a Cloudflare-protected site and anycast routing to the nearest edge PoP, adapted from Cloudflare’s Anycast and Anycast DNS diagrams.

or a public resolver such as 1.1.1.1) to translate the domain name into an IP address. Because the domain’s nameservers (NS records) point to Cloudflare, the DNS query eventually reaches Cloudflare’s authoritative DNS service [11].

Cloudflare’s DNS returns an IP address that belongs to Cloudflare, not to the origin server. These IP addresses are announced using anycast. This means that the same IP prefix is advertised from many Cloudflare data centres around the world. The Internet routing protocol BGP then directs the user’s packets to the “nearest” Cloudflare point of presence (PoP) according to the routing policy [9]. From the user’s perspective, they simply connect to one IP address, but the actual physical server that handles the connection depends on where they are on the network.

This mechanism shows how Cloudflare combines DNS and routing: DNS gives the client a Cloudflare IP, and anycast plus BGP select the concrete edge location. This is a concrete example of how DNS name resolution and network-layer routing work together to steer traffic to a nearby node.

3.2 Request Handling with Cloudflare

Once the TCP or QUIC connection reaches a Cloudflare edge server, Cloudflare behaves as the endpoint from the client’s point of view. It completes the TLS handshake with the browser and can speak HTTP/1.1, HTTP/2, or HTTP/3 depending on what the browser supports [7]. After the request is received, Cloudflare checks its edge cache to see whether it already has a fresh copy of the requested resource.

If the resource is found in the cache and is still valid, Cloudflare can respond immediately without contacting the origin server. This case is often marked as a cache hit in HTTP response headers. If the resource is not cached or is expired, Cloudflare acts as a reverse proxy: it opens a separate connection to the origin server, forwards the request, waits for the origin’s response, and then sends the response back to the client. At the same time, it can store parts of the response in its cache so that future requests from other users can be served more quickly [4, 2].

From the viewpoint of the textbook, this workflow combines several layers. At the application layer, Cloudflare terminates HTTP and manages cache headers. At the transport layer, it maintains separate connections to the client and to the origin, which can have different characteristics. At the network layer, it sits at the edge of its anycast network and forwards traffic deeper into the Internet

only when needed.

3.3 Additional Application-Layer Optimization

Beyond basic caching and proxy functions, Cloudflare also provides several application-layer optimisations. For example, it can compress responses using gzip or Brotli to reduce payload size, automatically convert and resize images to fit the client device, and reorder the loading of resources on a page to make the main content appear faster. Some features, such as “Early Hints” [5], allow Cloudflare to tell the browser about important resources (like CSS or fonts) before the full response from the origin has arrived, so that the browser can start fetching them in parallel.

Cloudflare also supports newer protocols such as HTTP/3 over QUIC, which can reduce connection setup time and handle packet loss more smoothly than traditional TCP.

4 Discussion and Analysis

When cached content is served from edge servers instead of the origin, the RTT between users and the data becomes shorter. This is especially important for users who are geographically far away from the origin data centre. Many small objects, such as images, JavaScript files, and style sheets, can be delivered directly from the edge, which also saves bandwidth on long-distance links. Anycast further helps by steering users to a nearby PoP without the user needing to know where that PoP is.

A second benefit is scalability for website operators. Instead of building their own global network, website owners can host their origin server in one region and let Cloudflare handle DNS, CDN, and reverse-proxy functions. When a site suddenly becomes popular, a large part of the extra traffic is absorbed by Cloudflare’s caches and network capacity. Cloudflare can also deploy support for new protocols, such as HTTP/2 or HTTP/3, once on its edge network, and many sites can instantly benefit without changing their own servers.

However, there are also trade-offs. The overall system becomes more complex. Now there are at least three logical components involved in each request: the client, the Cloudflare edge, and the origin server. Problems may appear at any of these points, and it can be harder to debug where exactly a delay or error comes from. The strong coupling between the application layer, the DNS layer, and the routing layer makes configuration mistakes more likely and sometimes harder to detect.

Another trade-off is dependence. When a site moves “behind Cloudflare”, it relies heavily on this single third-party provider. If Cloudflare has a large outage, as in my personal experience, many unrelated websites can be affected at the same time. Developers must also learn how Cloudflare’s cache, headers, and rules interact with their own application logic; otherwise they may see confusing behaviour such as “code has been updated but users still see the old version”.

References

- [1] M. Chapman. Cloudflare resolves outage that impacted thousands, chatgpt, x and more. <https://apnews.com/article/cloudflare-outage-x-openai-9335e8e0da2a0027d1fbac5eb97d11ae>, 2025. Accessed: 2025-11-21.
- [2] Cloudflare, Inc. Cloudflare cache responses. <https://developers.cloudflare.com/cache/concepts/cache-responses/>, 2025. Accessed: 2025-11-21.
- [3] Cloudflare, Inc. Cloudflare outage on november 18, 2025. <https://blog.cloudflare.com/18-november-2025-outage/>, 2025. Accessed: 2025-11-21.
- [4] Cloudflare, Inc. Default cache behavior. <https://developers.cloudflare.com/cache/concepts/default-cache-behavior/>, 2025. Accessed: 2025-11-21.
- [5] Cloudflare, Inc. Early hints: How cloudflare can improve website load times by 30 <https://blog.cloudflare.com/early-hints/>, 2025. Accessed: 2025-11-21.
- [6] Cloudflare, Inc. How cloudflare works. <https://developers.cloudflare.com/fundamentals/concepts/how-cloudflare-works/>, 2025. Accessed: 2025-11-21.
- [7] Cloudflare, Inc. Http/3: the past, the present, and the future. <https://blog.cloudflare.com/http3-the-past-present-and-future/>, 2025. Accessed: 2025-11-21.
- [8] Cloudflare, Inc. What is a content delivery network (cdn)? <https://www.cloudflare.com/learning/cdn/what-is-a-cdn/>, 2025. Accessed: 2025-11-21.
- [9] Cloudflare, Inc. What is anycast? | how does anycast work? <https://www.cloudflare.com/learning/cdn/glossary/anycast-network/>, 2025. Accessed: 2025-11-21.
- [10] Cloudflare, Inc. What is anycast dns? | how anycast works with dns. <https://www.cloudflare.com/learning/dns/what-is-anycast-dns/>, 2025. Accessed: 2025-11-21.
- [11] Cloudflare, Inc. What is dns? | how dns works. <https://www.cloudflare.com/learning/dns/what-is-dns/>, 2025. Accessed: 2025-11-21.
- [12] J. Kurose and K. Ross. Computer networks: A top down approach featuring the internet, 2010.